AuthService.java

```
package com.harisudhan.harisudhanmart.service;

import com.harisudhan.harisudhanmart.dao.UserDAO;
import com.harisudhan.harisudhanmart.exception.AuthException;
import com.harisudhan.harisudhanmart.exception.ValidationException;
import com.harisudhan.harisudhanmart.model.User;
import com.harisudhan.harisudhanmart.util.PasswordUtil;
import com.harisudhan.harisudhanmart.util.ValidationUtil;
import java.sql.SQLException;
import javax.servlet.http.HttpServletResponse;

/** Business rules for registration and login. No JDBC here — all persistence via UserDAO. */
public class AuthService {

    private final UserDAO userDAO;

    public AuthService(UserDAO userDAO) {
        this.userDAO = userDAO;
    }

    public User register(String name, String email, String password, String roleRaw)
            throws ValidationException, SQLException {
        if (ValidationUtil.isBlank(name)) {
            throw new ValidationException("name", "Name is required");
        }
        if (!ValidationUtil.isValidEmail(email)) {
            throw new ValidationException("email", "A valid email is required");
        }
        if (ValidationUtil.isBlank(password) || password.length() < 8) {
            throw new ValidationException("password", "Password must be at least 8 characters");
        }
        User.Role role;
        try {
            role = User.Role.valueOf(roleRaw == null ? "" : roleRaw.toUpperCase());
        } catch (IllegalArgumentException e) {
            throw new ValidationException("role", "Role must be BUYER or SELLER");
        }
        if (role == User.Role.ADMIN) {
            // Admin is seeded only; F1 forbids a public admin signup flow.
            throw new ValidationException("role", "Admin accounts cannot self-register");
        }
        if (userDAO.findByEmail(email).isPresent()) {
            throw new ValidationException("email", "An account with this email already exists");
        }

        User user = new User();
        user.setName(name.trim());
        user.setEmail(email.trim().toLowerCase());
        user.setPasswordHash(PasswordUtil.hash(password));
        user.setRole(role);
        return userDAO.create(user);
    }

    public User login(String email, String password) throws AuthException, SQLException {
        User user = userDAO.findByEmail(email == null ? "" : email.trim().toLowerCase())
                .orElseThrow(() -> new AuthException("Invalid email or password", HttpServletResponse.SC_UNAUTHORIZED));
        if (!PasswordUtil.matches(password == null ? "" : password, user.getPasswordHash())) {
            throw new AuthException("Invalid email or password", HttpServletResponse.SC_UNAUTHORIZED);
        }
        return user;
    }
}
```